04/02/2026: Classic Score Matching

Reference: Hyvärinen, A. (2005). *Estimation of non-normalized statistical models by score matching.*

CSCI 546: Diffusion Models

Table of contents

1. Course Logistics
2. Mini-Lecture: Diffusion
3. Group Work
4. Mini-Lecture: Score Matching
5. Lightning Chat

Course Logistics

Final Project Notes

Announcement

I've provided specs about the final project in the syllabus.

Reading Survey

1. Describe score matching. For example, how does it work and when would you use it?

Mini-Lecture: Diffusion

Mini-Lecture: Diffusion

See previous slide deck

Group Work

Group Work

See previous slide deck

Mini-Lecture: Score Matching

A fundamental problem

Let $\boldsymbol{X} \in \mathbb{R}^d$ be a random vector.

A fundamental problem

Let $\boldsymbol{X} \in \mathbb{R}^d$ be a random vector.

Any set of non-negative functions $\{q_{\boldsymbol{\theta}}(\boldsymbol{x})\}_{\boldsymbol{\theta} \in \Theta}$ parameterized by $\boldsymbol{\theta}$ can be used to construct $\{p_{\boldsymbol{\theta}}(\boldsymbol{x})\}_{\boldsymbol{\theta} \in \Theta}$, a family of probability densities over $\boldsymbol{X}$.

A fundamental problem

Let $\boldsymbol{X} \in \mathbb{R}^d$ be a random vector.

Any set of non-negative functions $\{q_{\boldsymbol{\theta}}(\boldsymbol{x})\}_{\boldsymbol{\theta} \in \Theta}$ parameterized by $\boldsymbol{\theta}$ can be used to construct $\{p_{\boldsymbol{\theta}}(\boldsymbol{x})\}_{\boldsymbol{\theta} \in \Theta}$, a family of probability densities over $\boldsymbol{X}$.

Since probability density functions **must** integrate to 1, we set

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \frac{q_{\boldsymbol{\theta}}(\boldsymbol{x})}{Z_{\boldsymbol{\theta}}},$$

where $Z_{\boldsymbol{\theta}} := \int q_{\boldsymbol{\theta}}(\boldsymbol{x}) d\boldsymbol{x}$ is a normalizing constant.

A fundamental problem

Let $\boldsymbol{X} \in \mathbb{R}^d$ be a random vector.

Any set of non-negative functions $\{q_{\boldsymbol{\theta}}(\boldsymbol{x})\}_{\boldsymbol{\theta} \in \Theta}$ parameterized by $\boldsymbol{\theta}$ can be used to construct $\{p_{\boldsymbol{\theta}}(\boldsymbol{x})\}_{\boldsymbol{\theta} \in \Theta}$, a family of probability densities over $\boldsymbol{X}$.

Since probability density functions **must** integrate to 1, we set

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \frac{q_{\boldsymbol{\theta}}(\boldsymbol{x})}{Z_{\boldsymbol{\theta}}},$$

where $Z_{\boldsymbol{\theta}} := \int q_{\boldsymbol{\theta}}(\boldsymbol{x}) d\boldsymbol{x}$ is a normalizing constant.

The problem. What if we can't compute $Z_{\boldsymbol{\theta}}$?

A fundamental problem

Let $\boldsymbol{X} \in \mathbb{R}^d$ be a random vector.

Any set of non-negative functions $\{q_{\boldsymbol{\theta}}(\boldsymbol{x})\}_{\boldsymbol{\theta} \in \Theta}$ parameterized by $\boldsymbol{\theta}$ can be used to construct $\{p_{\boldsymbol{\theta}}(\boldsymbol{x})\}_{\boldsymbol{\theta} \in \Theta}$, a family of probability densities over $\boldsymbol{X}$.

Since probability density functions **must** integrate to 1, we set

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \frac{q_{\boldsymbol{\theta}}(\boldsymbol{x})}{Z_{\boldsymbol{\theta}}},$$

where $Z_{\boldsymbol{\theta}} := \int q_{\boldsymbol{\theta}}(\boldsymbol{x}) d\boldsymbol{x}$ is a normalizing constant.

The problem. What if we can't compute $Z_{\boldsymbol{\theta}}$?

A solution. We show how to train and sample from these models using score matching, which matches the **score functions** of the model to the data.

Score function

Definition. The **score function** (or *Stein score*) is the gradient of the log likelihood with respect to the data vector $\boldsymbol{x}$:

$$\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})$$

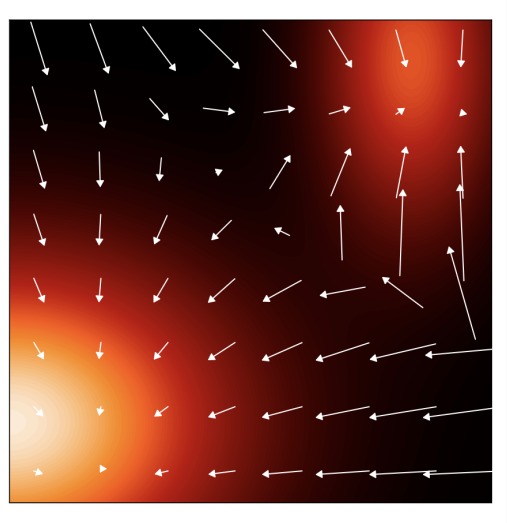


Figure 1: Illustration of score function for a mixture of two-dimensional Gaussians.

Score function

Definition. The **score function** (or *Stein score*) is the gradient of the log likelihood with respect to the data vector $\boldsymbol{x}$:

$$\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})$$

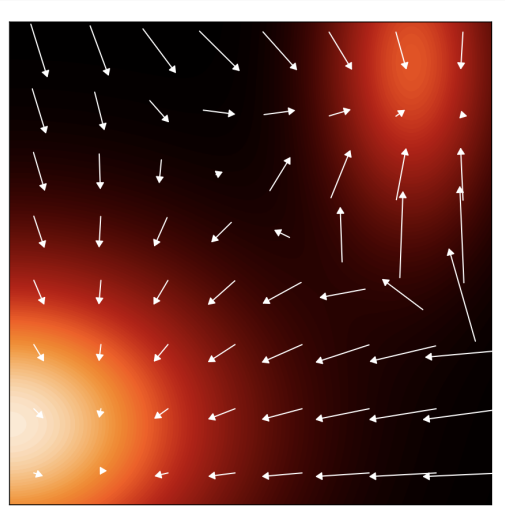


Figure 1: Illustration of score function for a mixture of two-dimensional Gaussians.

Poll. Interpretation?

Score function

Definition. The **score function** (or *Stein score*) is the gradient of the log likelihood with respect to the data vector $\boldsymbol{x}$:

$$\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})$$

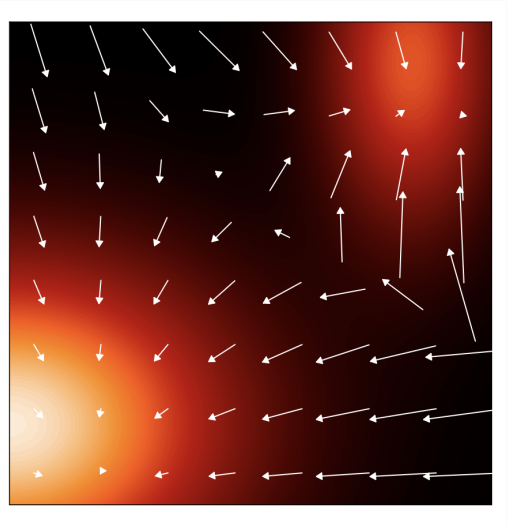


Figure 1: Illustration of score function for a mixture of two-dimensional Gaussians.

Poll. Interpretation? The score function is a vector field pointing to the direction of steepest increase in the log likelihood.

Score function

Definition. The **score function** (or *Stein score*) is the gradient of the log likelihood with respect to the data vector $\boldsymbol{x}$:

$$\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})$$

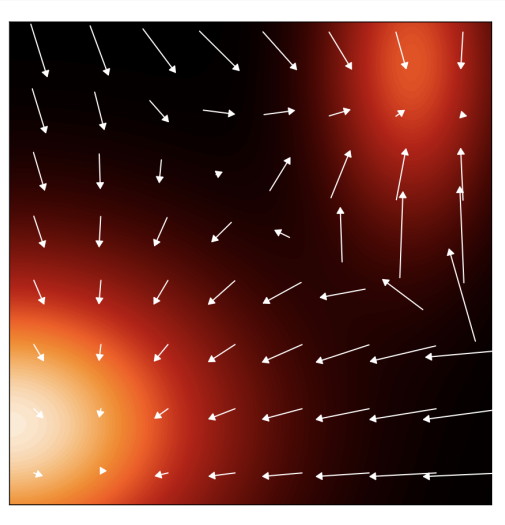


Figure 1: Illustration of score function for a mixture of two-dimensional Gaussians.

Poll. Why is this useful?

Score function

Definition. The **score function** (or *Stein score*) is the gradient of the log likelihood with respect to the data vector $\boldsymbol{x}$:

$$\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})$$

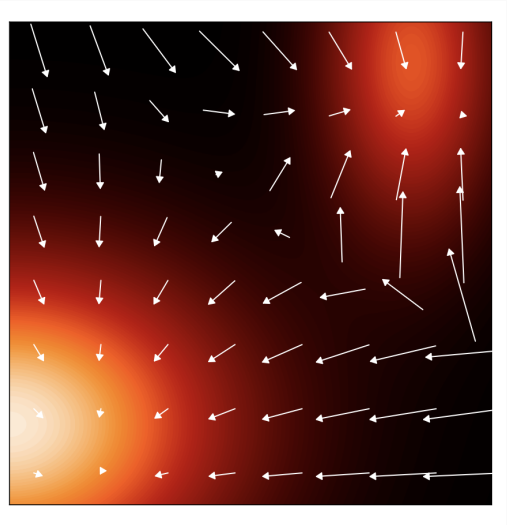


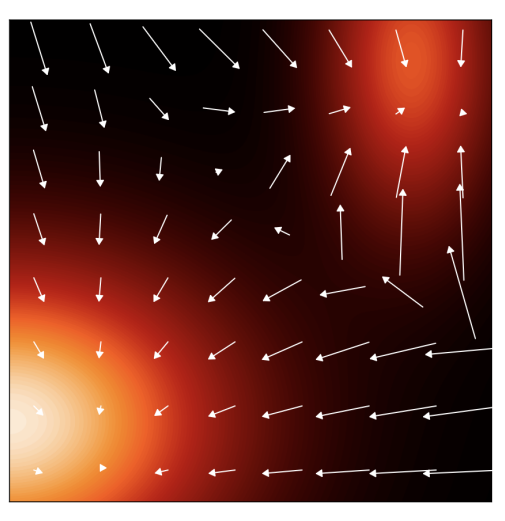
Figure 1: Illustration of score function for a mixture of two-dimensional Gaussians.

Poll. Why is this useful? We don't need the normalizing constant!

$$\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln q_{\boldsymbol{\theta}}(\boldsymbol{x}) - \cancel{\nabla_{\boldsymbol{x}} \ln Z_{\boldsymbol{\theta}}} = \nabla_{\boldsymbol{x}} \ln q_{\boldsymbol{\theta}}(\boldsymbol{x}).$$

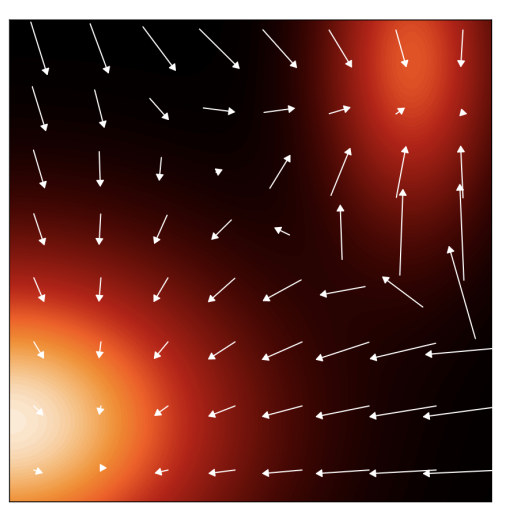
Langevin Sampling

Poll. How can we use the score function to obtain a sample $\boldsymbol{x}$ from the probability distribution with density p?



Langevin Sampling

Poll. How can we use the score function to obtain a sample $\boldsymbol{x}$ from the probability distribution with density p?



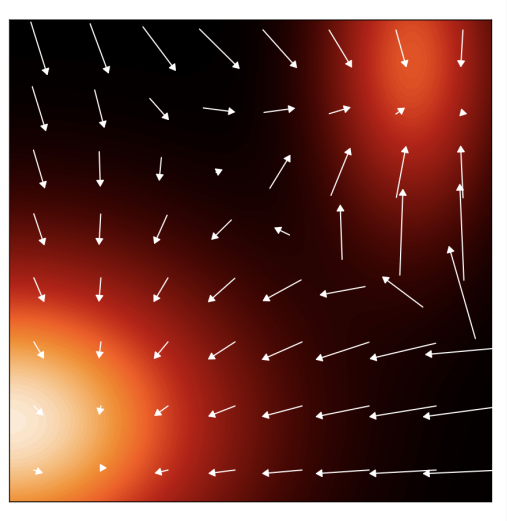
Solution. Start with an initial value $\boldsymbol{x}^{(0)}$, and then iterate the following Markov chain steps:

$$\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \eta \underbrace{\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})}_{\text{score function}} + \sqrt{2\eta}\boldsymbol{\epsilon}^{(t)}, \qquad t = 1, 2, \ldots T.$$

where $\boldsymbol{\epsilon}^{(t)} \sim \mathcal{N}(0, \boldsymbol{I})$ and η is a learning rate.

Langevin Sampling

Poll. How can we use the score function to obtain a sample $\boldsymbol{x}$ from the probability distribution with density p?



Solution. Start with an initial value $\boldsymbol{x}^{(0)}$, and then iterate the following Markov chain steps:

$$\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \eta \underbrace{\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})}_{\text{score function}} + \sqrt{2\eta}\boldsymbol{\epsilon}^{(t)}, \qquad t = 1, 2, \ldots T.$$

where $\boldsymbol{\epsilon}^{(t)} \sim \mathcal{N}(0, \boldsymbol{I})$ and η is a learning rate.

Theorem. As $T \to \infty$, $\boldsymbol{x}^{(T)}$ becomes an independent sample from $p_{\boldsymbol{\theta}}(\boldsymbol{x})$.

Langevin Sampling

Langevin Sampling. Start with an initial value $\boldsymbol{x}^{(0)}$, and then iterate the following Markov chain steps:

$$\boxed{\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \eta \, \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x}) + \sqrt{2\eta}\boldsymbol{\epsilon}^{(t)}, \qquad t = 1, 2, \ldots T.}$$

Langevin Sampling

Langevin Sampling. Start with an initial value $\boldsymbol{x}^{(0)}$, and then iterate the following Markov chain steps:

$$\boxed{\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \eta \, \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x}) + \sqrt{2\eta}\boldsymbol{\epsilon}^{(t)}, \qquad t = 1, 2, \ldots T.}$$

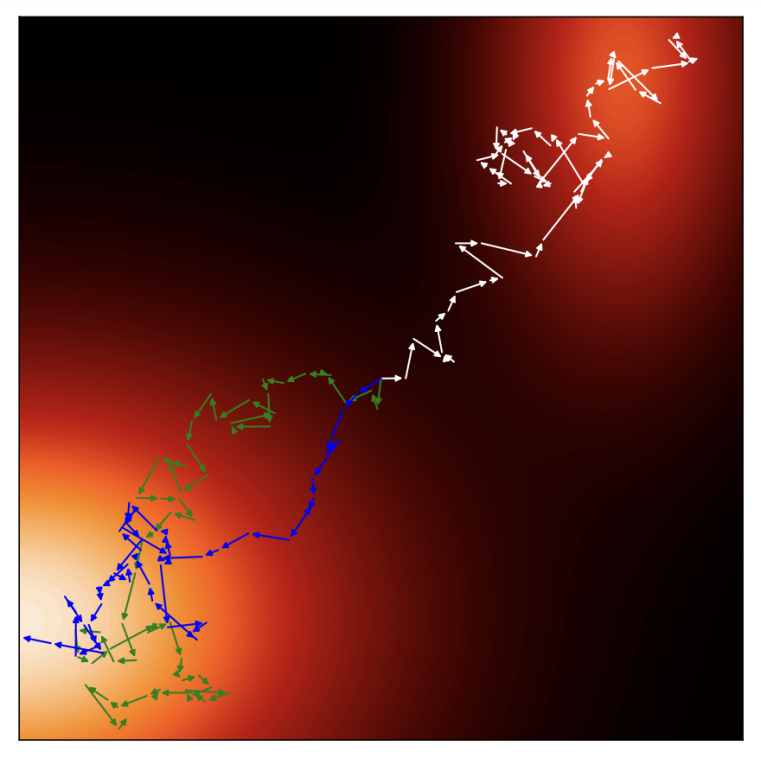


Figure 2: Three trajectories, all starting from the center of the plot.

Langevin Sampling

Langevin Sampling. Start with an initial value $\boldsymbol{x}^{(0)}$, and then iterate the following Markov chain steps:

$$\boxed{\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \eta \, \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x}) + \sqrt{2\eta}\boldsymbol{\epsilon}^{(t)}, \qquad t = 1, 2, \ldots T.}$$

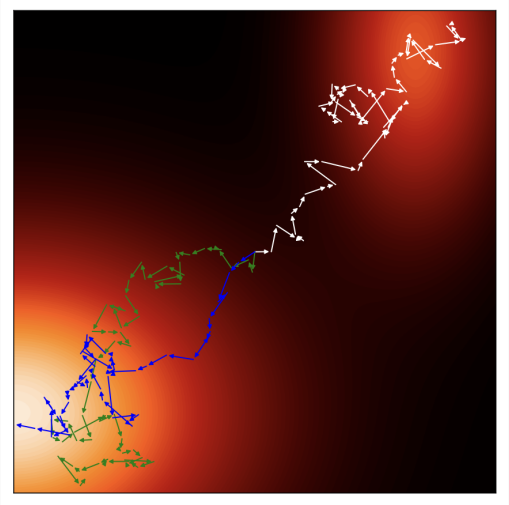


Figure 2: Three trajectories, all starting from the center of the plot.

Poll. Why is this useful?

Langevin Sampling

Langevin Sampling. Start with an initial value $\boldsymbol{x}^{(0)}$, and then iterate the following Markov chain steps:

$$\boxed{\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \eta \, \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x}) + \sqrt{2\eta}\boldsymbol{\epsilon}^{(t)}, \qquad t = 1, 2, \ldots T.}$$

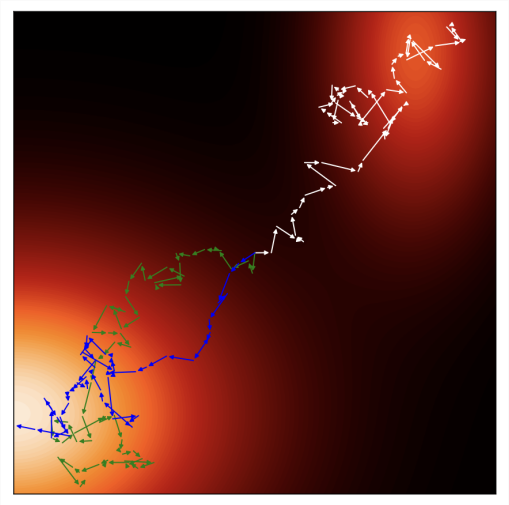


Figure 2: Three trajectories, all starting from the center of the plot.

Poll. Why is this useful?

Solution. We don't need the probability distribution to be normalized!

Langevin Sampling

Langevin Sampling. Start with an initial value $\boldsymbol{x}^{(0)}$, and then iterate the following Markov chain steps:

$$\boxed{\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \eta \, \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x}) + \sqrt{2\eta}\boldsymbol{\epsilon}^{(t)}, \qquad t = 1, 2, \ldots T.}$$

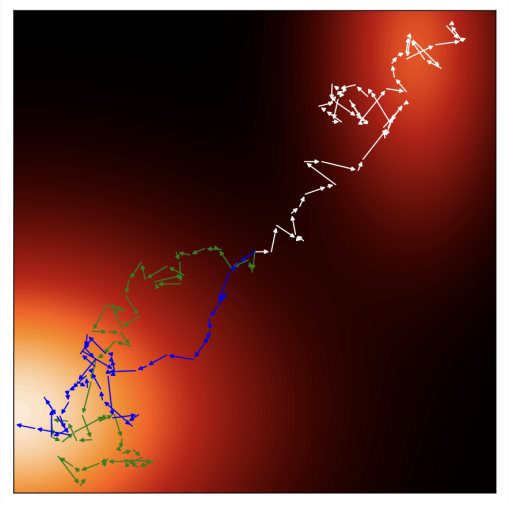


Figure 2: Three trajectories, all starting from the center of the plot.

Problem. But how do we learn $\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})$ from data?

Langevin Sampling

Langevin Sampling. Start with an initial value $\boldsymbol{x}^{(0)}$, and then iterate the following Markov chain steps:

$$\boxed{\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \eta \, \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x}) + \sqrt{2\eta}\boldsymbol{\epsilon}^{(t)}, \qquad t = 1, 2, \ldots T.}$$

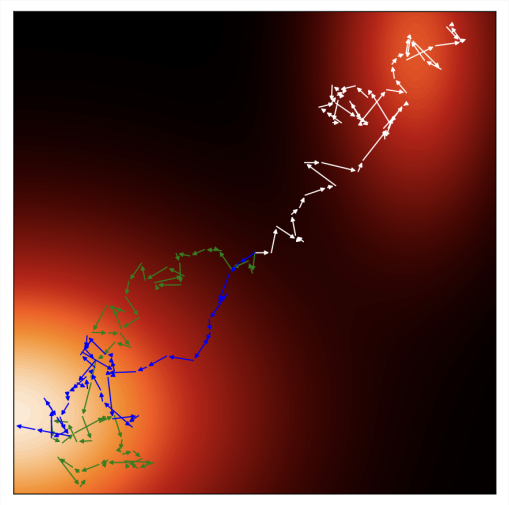


Figure 2: Three trajectories, all starting from the center of the plot.

Problem. But how do we learn $\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})$ from data?

Solution. `Score matching` to the rescue!

Score Matching

Let

- $p_0(\boldsymbol{x})$ be the density of the true data generating distribution
- $p_{\boldsymbol{\theta}}(\boldsymbol{x})$ be the density of our model.

Score Matching

Then the **score matching** training loss is given by

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x}$$

Score Matching

Then the **score matching** training loss is given by

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x}$$

Poll. What is the interpretation of this training loss?

Score Matching

Then the **score matching** training loss is given by

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x}$$

Poll. What is the interpretation of this training loss?

Solution. It's the expected squared error between the model score and the true score. (Multiplied by a half).

Score Matching

Then the **score matching** training loss is given by

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x}$$

Idea. If we want to learn a modeled score function in a flexible manner, we could train a neural network $s(\boldsymbol{x}, \boldsymbol{\theta}) = \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})$ within this loss function.

Problem. There's a blocker to doing this. What is it?

Score Matching

Then the **score matching** training loss is given by

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{\nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x}$$

Idea. If we want to learn a modeled score function in a flexible manner, we could train a neural network $s(\boldsymbol{x}, \boldsymbol{\theta}) = \nabla_{\boldsymbol{x}} \ln p_{\boldsymbol{\theta}}(\boldsymbol{x})$ within this loss function.

Problem. There's a blocker to doing this. What is it? Even if we have collected a bunch of observations $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots \boldsymbol{x}_n$, there's no way to know the true score of these observations!

Hyvärinen's Solution

Problem. We want to train a neural network $s(\boldsymbol{x}, \boldsymbol{\theta})$ via the loss function

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{s(\boldsymbol{x}, \boldsymbol{\theta})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

but we don't know the true score .

Hyvärinen's Solution

Problem. We want to train a neural network $s(\boldsymbol{x},\boldsymbol{\theta})$ via the loss function

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{s(\boldsymbol{x},\boldsymbol{\theta})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

but we don't know the true score .

Solution. Using integration by parts, we can rewrite this as

$$J(\boldsymbol{\theta}) = \int \left\{ \nabla \cdot s(\boldsymbol{x},\boldsymbol{\theta}) + \frac{1}{2} \| s(\boldsymbol{x},\boldsymbol{\theta}) \|^2 \right\} p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

where $\nabla \cdot s = \sum_{i=1}^{d} \frac{\partial s_i}{\partial x_i}$ is the divergence operator.

Hyvärinen's Solution

Problem. We want to train a neural network $s(\boldsymbol{x},\boldsymbol{\theta})$ via the loss function

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{s(\boldsymbol{x},\boldsymbol{\theta})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

but we don't know the true score .

Solution. Using integration by parts, we can rewrite this as

$$J(\boldsymbol{\theta}) = \int \left\{ \nabla \cdot s(\boldsymbol{x},\boldsymbol{\theta}) + \frac{1}{2} \| s(\boldsymbol{x},\boldsymbol{\theta}) \|^2 \right\} p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

where $\nabla \cdot s = \sum_{i=1}^{d} \frac{\partial s_i}{\partial x_i}$ is the divergence operator.

Implication. Who cares?

Hyvärinen's Solution

Problem. We want to train a neural network $s(\boldsymbol{x},\boldsymbol{\theta})$ via the loss function

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{s(\boldsymbol{x},\boldsymbol{\theta})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

but we don't know the true score .

Solution. Using integration by parts, we can rewrite this as

$$J(\boldsymbol{\theta}) = \int \left\{ \nabla \cdot s(\boldsymbol{x},\boldsymbol{\theta}) + \frac{1}{2} \| s(\boldsymbol{x},\boldsymbol{\theta}) \|^2 \right\} p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

where $\nabla \cdot s = \sum_{i=1}^{d} \frac{\partial s_i}{\partial x_i}$ is the divergence operator.

Implication. Who cares? Now the integrand doesn't refer to the true score at all!

Hyvärinen's Solution

Problem. We want to train a neural network $s(\boldsymbol{x},\boldsymbol{\theta})$ via the loss function

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{s(\boldsymbol{x},\boldsymbol{\theta})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

but we don't know the true score .

Solution. Using integration by parts, we can rewrite this as

$$J(\boldsymbol{\theta}) = \int \left\{ \nabla \cdot s(\boldsymbol{x},\boldsymbol{\theta}) + \frac{1}{2} \| s(\boldsymbol{x},\boldsymbol{\theta}) \|^2 \right\} p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

where $\nabla \cdot s = \sum_{i=1}^{d} \frac{\partial s_i}{\partial x_i}$ is the divergence operator.

Implementation. But we also don't know $p_0(\boldsymbol{x})$. So how to use in practice?

Hyvärinen's Solution

Problem. We want to train a neural network $s(\boldsymbol{x},\boldsymbol{\theta})$ via the loss function

$$J(\boldsymbol{\theta}) = \int \frac{1}{2} \| \underbrace{s(\boldsymbol{x},\boldsymbol{\theta})}_{\text{model score}} - \underbrace{\nabla_{\boldsymbol{x}} \ln p_0(\boldsymbol{x})}_{\text{true score}} \|^2 \, p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

but we don't know the true score .

Solution. Using integration by parts, we can rewrite this as

$$J(\boldsymbol{\theta}) = \int \left\{ \nabla \cdot s(\boldsymbol{x},\boldsymbol{\theta}) + \frac{1}{2} \| s(\boldsymbol{x},\boldsymbol{\theta}) \|^2 \right\} p_0(\boldsymbol{x}) \, d\boldsymbol{x},$$

where $\nabla \cdot s = \sum_{i=1}^{d} \frac{\partial s_i}{\partial x_i}$ is the divergence operator.

Implementation. But we also don't know $p_0(\boldsymbol{x})$. So how to use in practice? If we have N observations from $p_0(\boldsymbol{x})$, then we can use a Monte Carlo approximation:

$$J(\boldsymbol{\theta}) \approx \frac{1}{N} \sum_{i=1}^{N} \left\{ \nabla \cdot s(\boldsymbol{x}_i,\boldsymbol{\theta}) + \frac{1}{2} \| s(\boldsymbol{x}_i,\boldsymbol{\theta}) \|^2 \right\}$$

Lightning Chat

Lightning Chat